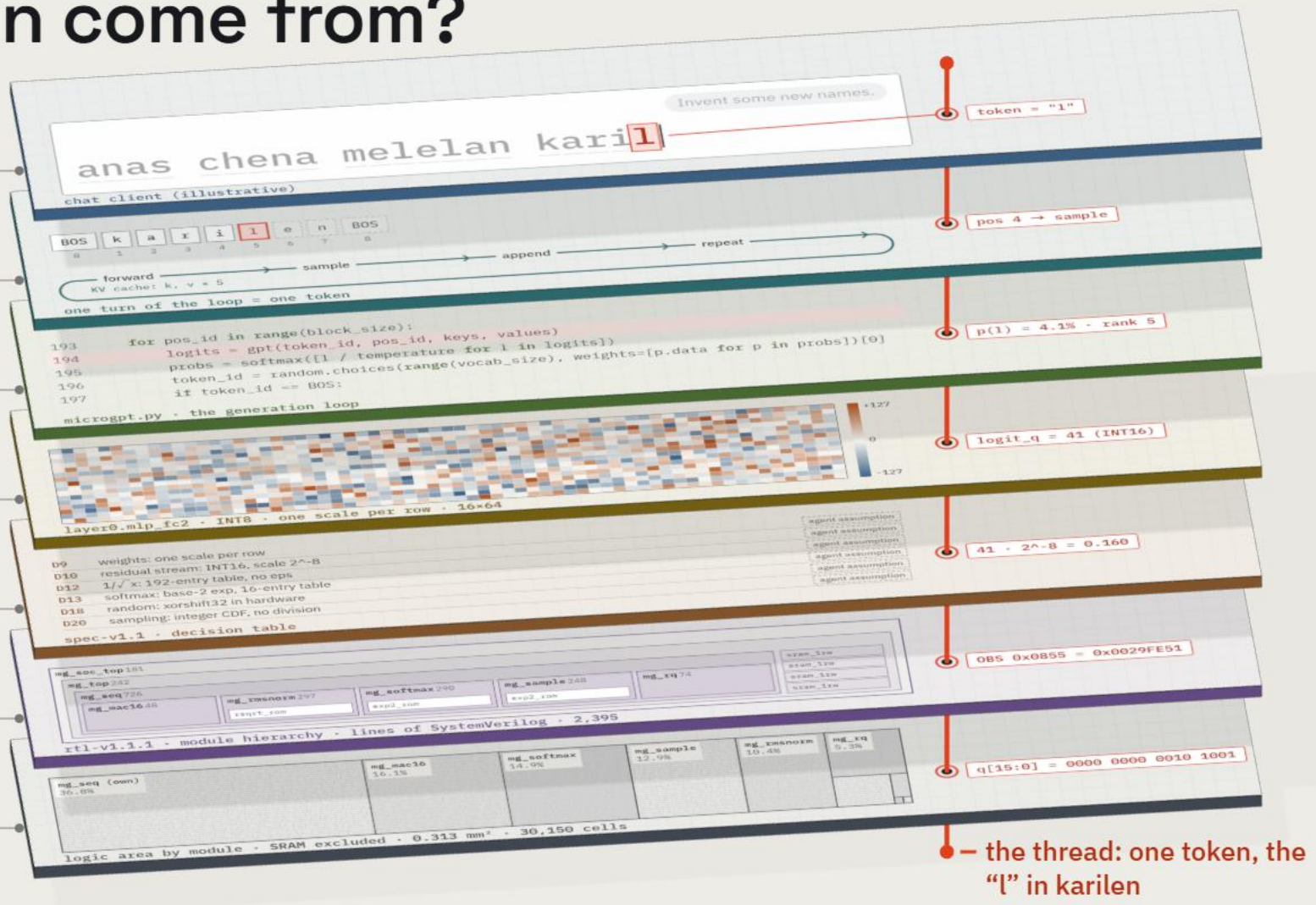
not taped out · RTL simulation + sky130 synthesis

Where does a token come from?

- Service
- Loop
- Python
- Integer math
- Spec
- RTL
- Cells

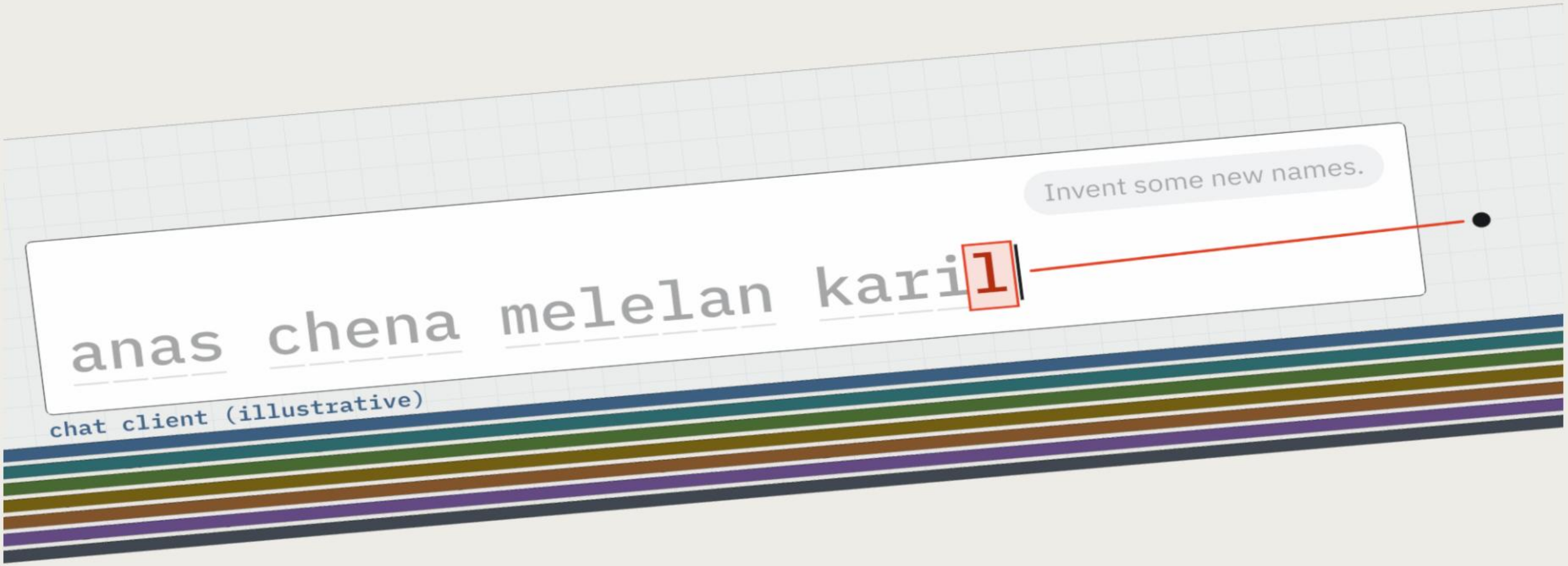


One token, traced from a chat service down to standard cells, and how three AI agent teams built the hardware.

A chat answers one token at a time

1 token = 1 letter

illustration · names from the RTL simulation



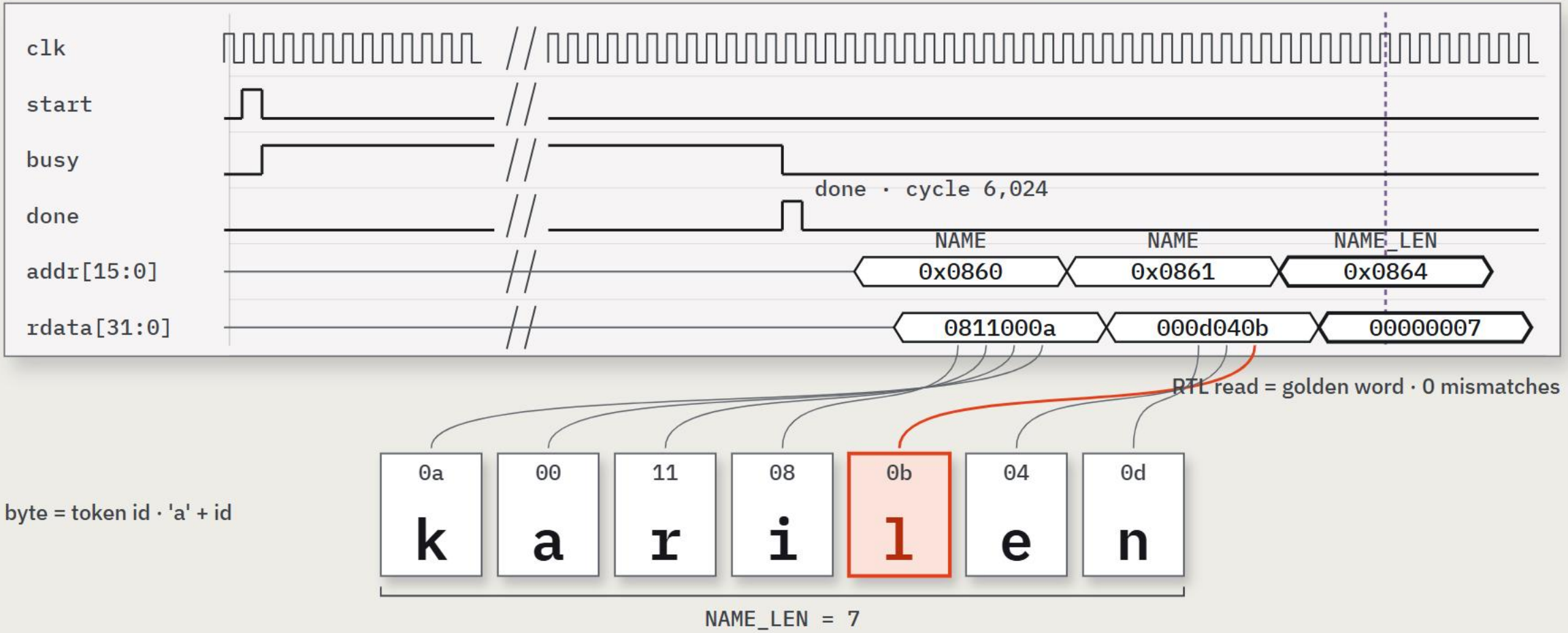
Freeze on one token and ask where it came from. The chat window is an illustration; the names in it are real outputs of the RTL.

source names: RTL simulation log (facts names)

This name came out of RTL simulation, not a GPU

RTL simulation `mg_top · icarus · arch-golden-v1.0 · s2a-n03`

not taped out · RTL simulation + sky130 synthesis



not a GPU · one logic block `3 agent teams` **spec · design · verify** **< 20 h**

karilen, read back from the design's observation memory. RTL simulation and sky130 synthesis only; nothing was taped out.

`source RTL test log NAME / NAME_LEN words`

not taped out · RTL simulation + sky130 synthesis

Behind almost every LLM service is one loop

- Service
- **Loop**
  forward · sample · append
- Python
- Integer math
- Spec
- RTL
- Cells

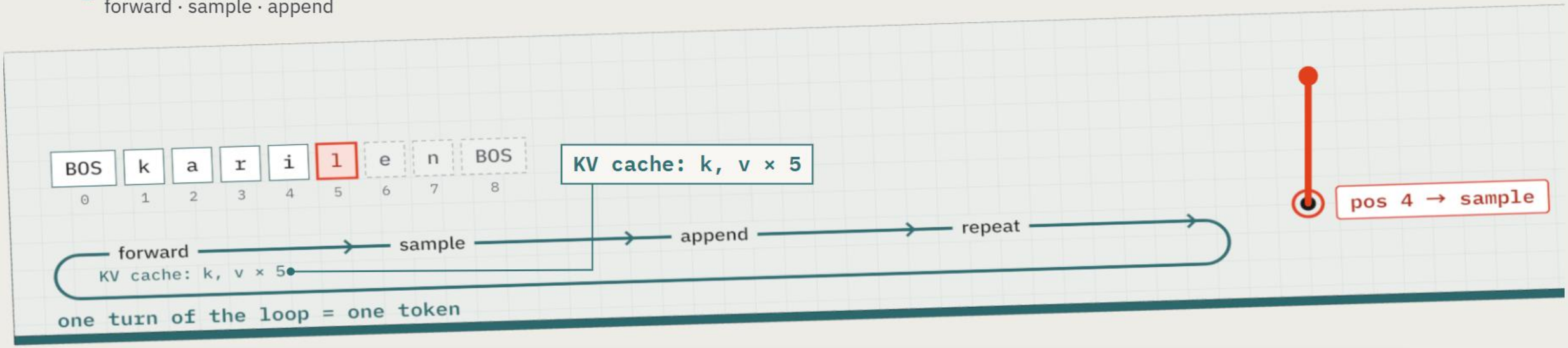


built by **3 agent teams + 1 human** checks 2 simulators · Yosys checked by **verif (independent)** human design decisions □ **0**

One turn of the loop yields exactly one token

Loop

forward · sample · append



microgpt.py · 191–199

tokenize
forward
probabilities
sample
stop
append

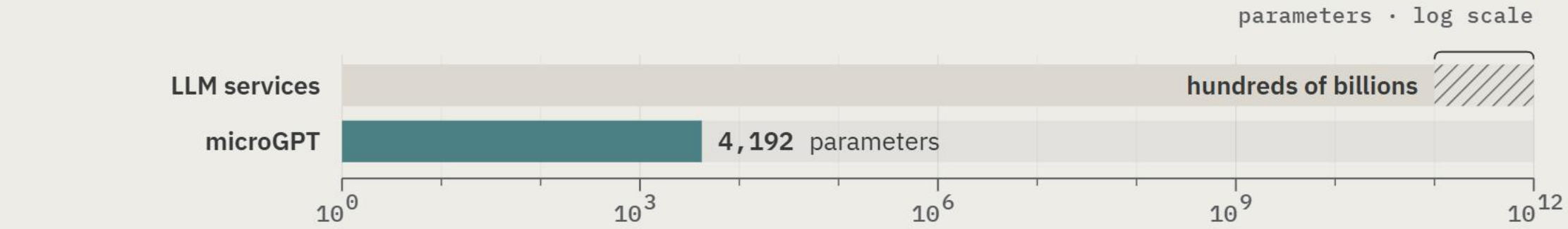
```
191  token_id = BOS
192  sample = []
193  for pos_id in range(block_size):
194      logits = gpt(token_id, pos_id, keys, values)
195      probs = softmax([l / temperature for l in logits])
196      token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
197      if token_id == BOS:
198          break
199      sample.append(uchars[token_id])                                    "kari" + "l"
```

Tokenize, run a forward pass, get a probability for every next token, sample one, append it, repeat.

source microgpt.py generate loop

The same loop runs at 4,192 parameters

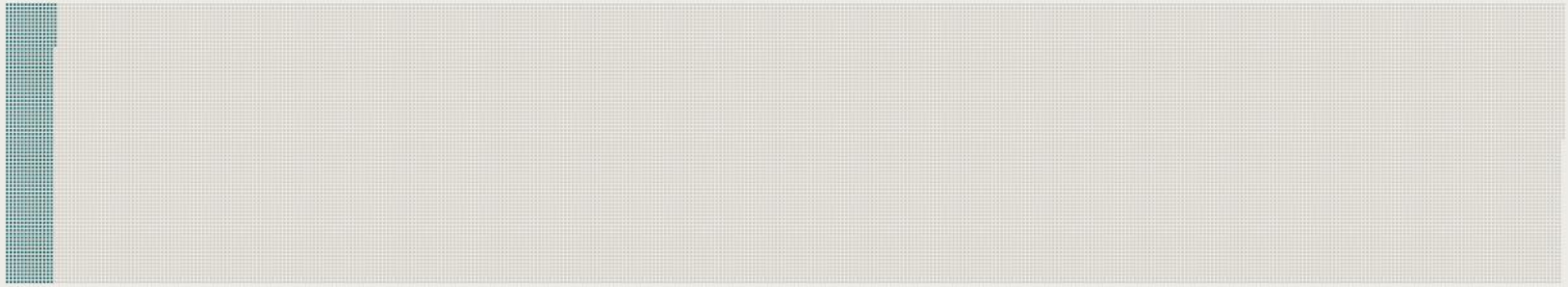
Loop
forward · sample · append



1	16	4	27	16
n_layer	n_embd	n_head	26 + BOS	block_size
layer	channels	attention heads	tokens	context positions
microgpt.py:75	microgpt.py:76	microgpt.py:78	microgpt.py:26	microgpt.py:77

1,000 names trained on microgpt.py:152

32,033 names in the list



new names, not in the list

chena
melelan
karilen
janan

One layer, 16 channels, 4 heads, 27 tokens. Trained on 1,000 of 32,033 names; production models run the same loop at hundreds of billions.

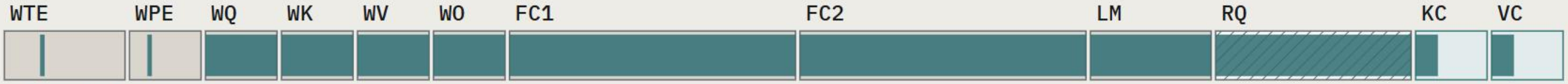
source facts.json model

Every new token rereads all weights and the growing cache

Loop
forward · sample · append



spec-v1.1 · M · weights + KV cache



WTE · WPE: one row

5 / 16

read for one token (position 4) · ≈ 1 multiply per byte read



Each position leaves a key and a value; later positions only read them. That is why generation is mostly a memory problem.

source spec section M: KV row layout

not taped out · RTL simulation + sky130 synthesis

The reference model is one Python file in float

Service

Loop

Python
199 lines, float

Integer math

Spec

RTL

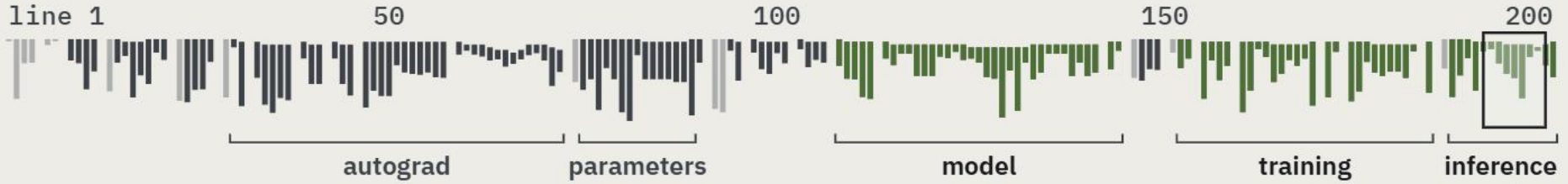
Cells



built by **3 agent teams + 1 human** checks 2 simulators · Yosys checked by **verif (independent)** human design decisions □ **0**

The whole GPT, training included, is one plain file

microgpt.py · one file · pure Python · float



- Service
- Loop
- **Python**
- Integer math
- Spec
- RTL
- Cells

```
190     keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
191     token_id = BOS
192     sample = []
193     for pos_id in range(block_size):
194         logits = gpt(token_id, pos_id, keys, values)
195         probs = softmax([l / temperature for l in logits])
196         token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
197         if token_id == BOS:
```

microgpt.py · the generation loop

source Karpathy · microgpt.py · gist 8627fe009c40f57531cb18360106ce95

About two hundred lines of plain Python, no libraries. Every later layer is a translation of this file.

source Karpathy, microgpt.py

One position costs up to 4,016 multiplications

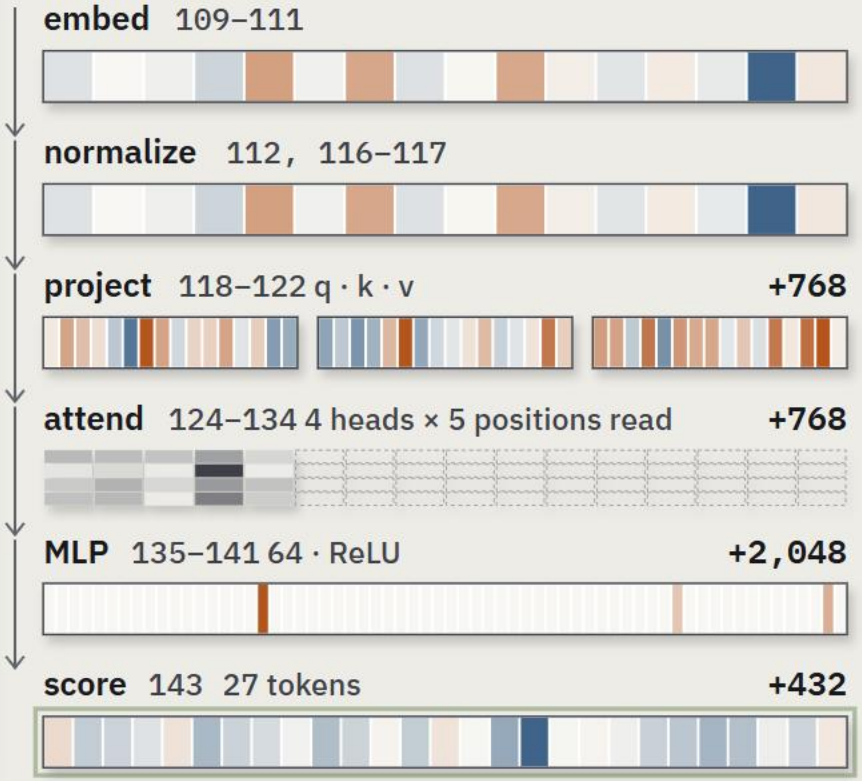
one position of the forward pass · gpt() · microgpt.py 108–144

```
123         x_attn = []
124         for h in range(n_head):
125             hs = h * head_dim
126             q_h = q[hs:hs+head_dim]
127             k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
128             v_h = [vi[hs:hs+head_dim] for vi in values[li]]
129             attn_logits = [sum(q_h[j] * k_h[t][j] for j in rang…
130             attn_weights = softmax(attn_logits)
131             head_out = [sum(attn_weights[t] * v_h[t][j] for t i…
132             x_attn.extend(head_out)
133         x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
134         x = [a + b for a, b in zip(x, x_residual)]
135         # 2) MLP block
136         x_residual = x
137         x = rmsnorm(x)
138         x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
139         x = [xi.relu() for xi in x]
140         x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
141         x = [a + b for a, b in zip(x, x_residual)]
142
143     logits = linear(x, state_dict['lm_head'])
144     return logits
```

multiplies per position

4,016

3504 + 32(p+1)
max · p = 15



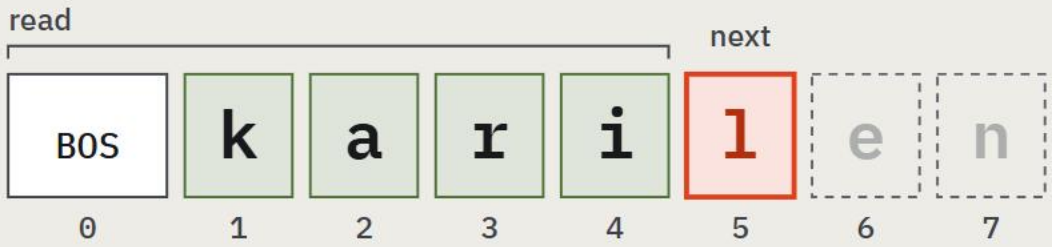
tool lab/instrumented.py · 118 × gpt()

Embed, normalize, project to q, k, v, attend, run a small MLP, score all 27 tokens.

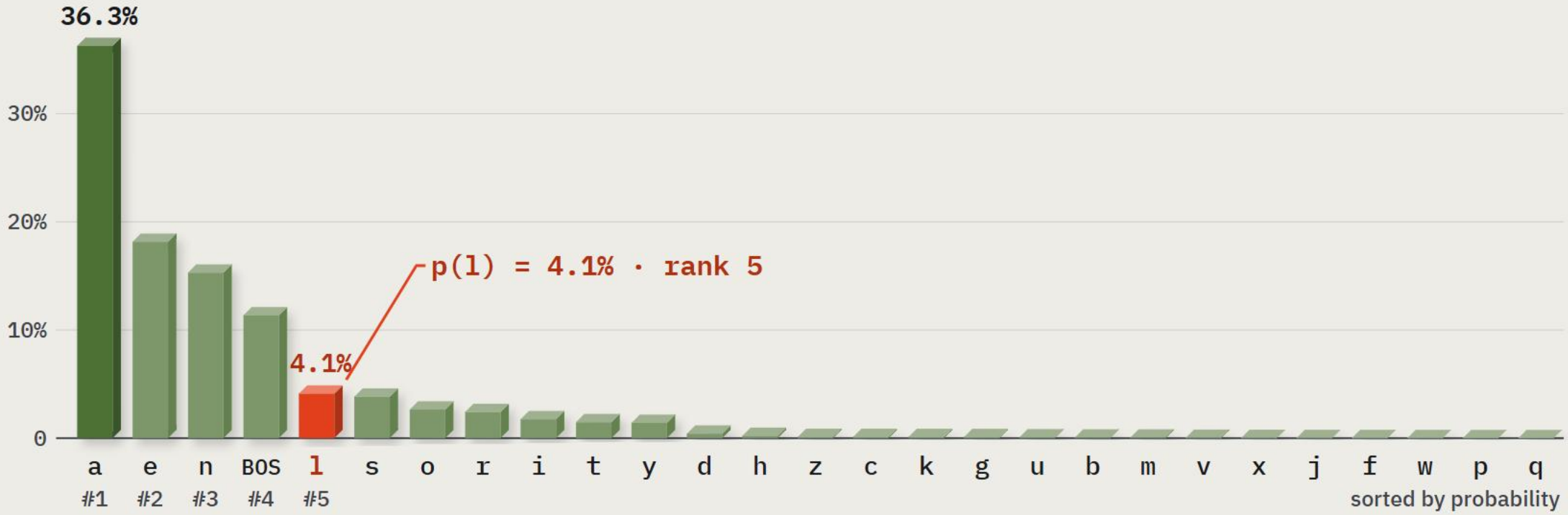
source facts.json model (instrumented run, lab/instrumented.py)

Our letter ranked fifth, at 4.1%

next-letter probabilities · float model · softmax(logits / T) · T = 0.5



karilen



After k, a, r, i the favourite next letter is a. Our letter l is fifth, so sampling has real work to do.

source facts.json thread (float)

Act 3 · Integer math

Silicon wants integers, so every number became one

not taped out · RTL simulation + sky130 synthesis

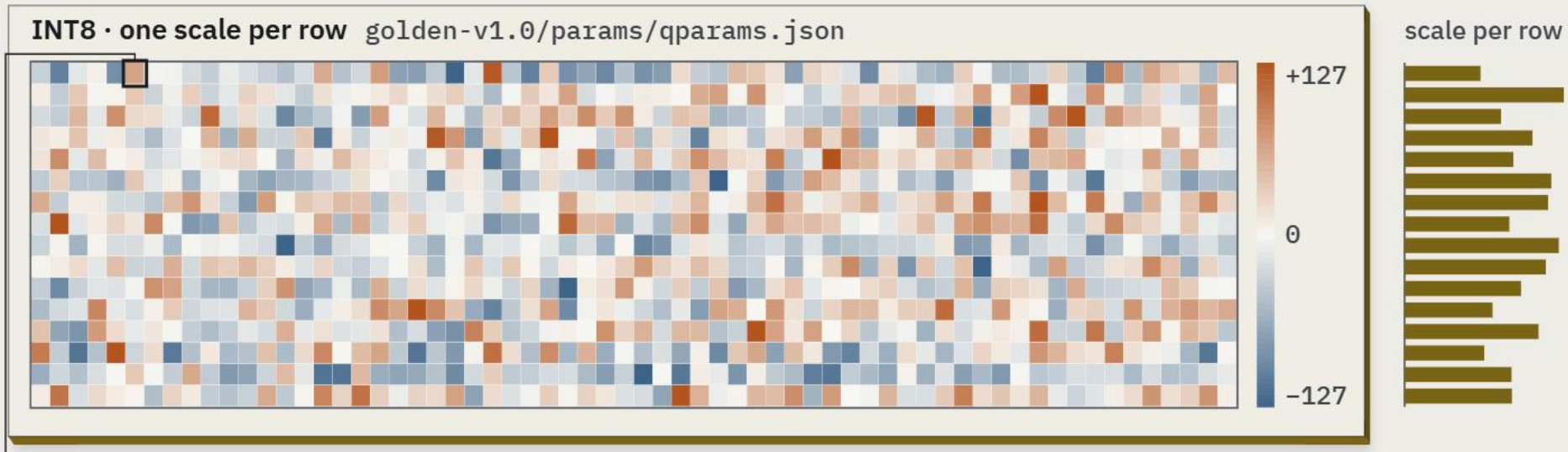
- Service
- Loop
- Python
- **Integer math**
  INT8 weights, fixed scales
- Spec
- RTL
- Cells



built by **3 agent teams + 1 human** checks 2 simulators · Yosys checked by **verif (independent)** human design decisions □ **0**

Weights became eight-bit, with one scale per row

weights · layer0.mlp_fc2 · 16 × 64



- Service
- Loop
- Python
- **Integer math**
- Spec
- RTL
- Cells

residual stream · INT16 · × 2^-8 thread position

−159	167	−132	−279	507	49	502	−61	53	312	22	−23	−23	33	−736	230

built by **arch agent** tool lab/m2_lab.py

The real layer0.mlp_fc2 matrix, before and after quantization. The residual stream became sixteen-bit.

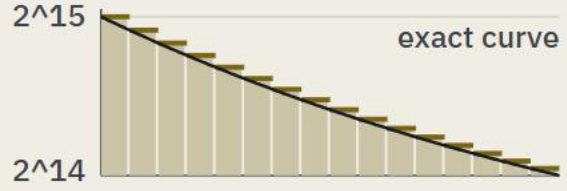
source weights.json, golden qparams (lab/m2_lab.py)

Hard functions became tables of 16 and 192 entries

hard functions → tables · golden-v1.0/luts

exponential softmax · sampler

round(2^15 · 2^(−f/16))



f 0	8000	7a93
f 2	7560	7066
f 4	6ba2	6712
f 6	62b4	5e84
f 8	5a82	56ac
f 10	52ff	4f7b
f 12	4c1c	48e2
f 14	45cb	42d5

luts/exp2_f4.hex — entries **16**

1 / square root RMSNorm

round(2^18 / √m) · m = 64 … 255



64	8000	7f03	7e0c	7d1a	7c2e	7b46	7a64	7987	78ae	77da	770a	763e	7576	74b2	73f2	7336
80	727d	71c7	7115	7066	6fba	6f12	6e6c	6dc9	6d29	6c8b	6bf0	6b58	6ac2	6a2f	699e	690f
96	6883	67f9	6771	66ea	6666	65e4	6564	64e6	6469	63ef	6376	62fe	6289	6215	61a2	6132
112	60c2	6054	5fe8	5f7d	5f13	5eab	5e44	5ddf	5d7a	5d17	5cb5	5c55	5bf5	5b97	5b3a	5ade
128	5a82	5a28	59d0	5978	5921	58cb	5876	5822	57cf	577c	572b	56db	568b	563d	55ef	55a2
144	5555	550a	54bf	5475	542c	53e4	539c	5355	530f	52c9	5284	5240	51fc	51b9	5177	5135
160	50f4	50b4	5074	5035	4ff6	4fb8	4f7a	4f3d	4f01	4ec5	4e8a	4e4f	4e14	4dda	4da1	4d68
176	4d30	4cf8	4cc1	4c8a	4c53	4c1d	4be7	4bb2	4b7e	4b49	4b15	4ae2	4aaf	4a7c	4a4a	4a18
192	49e7	49b6	4985	4955	4925	48f5	48c6	4897	4868	483a	480c	47df	47b2	4785	4758	472c
208	4700	46d5	46aa	467f	4654	462a	4600	45d6	45ad	4583	455b	4532	450a	44e2	44ba	4492
224	446b	4444	441e	43f7	43d1	43ab	4385	4360	433b	4316	42f1	42cc	42a8	4284	4260	423d
240	4219	41f6	41d3	41b1	418e	416c	414a	4128	4106	40e5	40c3	40a2	4082	4061	4040	4020

luts/rsqrt8.hex — entries **192**

The softmax exponential is a base-2 table; one over a square root is a 192-entry table. Values from the golden ROM files.

source golden exp2_lut, rsqrt_lut

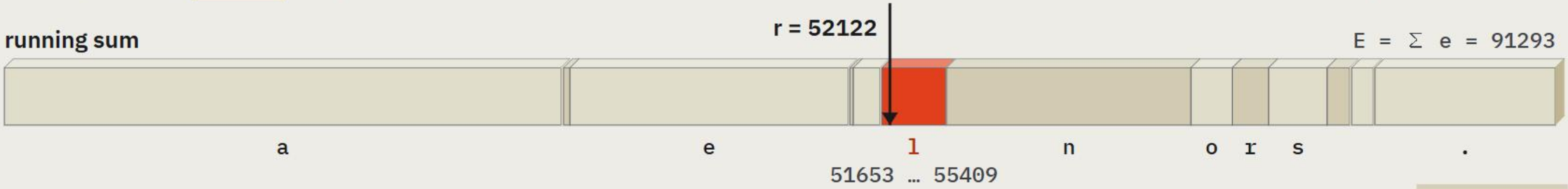
Even the dice are integers, so the names can match

the dice · sample_q · xorshift32 · seed 42



r = (x · E) >> 32 = (2452169066 · 91293) >> 32 = 52122

sampler **no division**



one draw per token

RTL = golden · 44 names · **mismatch 0**



A 32-bit xorshift and an integer running sum pick the token, with no division in the sampler. Design and golden model draw the same numbers in the same order.

source facts.json thread (integer sampling)

Integer math cost almost nothing in accuracy

the cost · float → integer

held-out loss change 2,000 names

+0.0019%

NLL 2.3786 → 2.3786

0.0019% budget 3%

same top token as float

99.54%

float · Python



logit **0.158** probability **4.1%** rank **5**

integer · INT16 logit_q = 41 (INT16)



logit **41 · 2^-8 = 0.160** probability **4.1%** rank **5**

human design decisions ☐ 0 spec decisions: agent assumptions, tested

On 2,000 held-out names the loss moved +0.0019%, and the top token matched the float model 99.54% of the time.

source facts.json integer, thread

not taped out · RTL simulation + sky130 synthesis

Every one of those choices is written in one spec

- Service
- Loop
- Python
- Integer math
- **Spec**
  the contract both teams build to
- RTL
- Cells



built by **3 agent teams + 1 human** checks 2 simulators · Yosys checked by **verif (independent)** human design decisions □ **0**

Each decision row keeps its reason and the rejected option

spec-v1.1/SPEC.md · 396 lines · D decisions :358

English translation of the Korean original

every row: agent assumption

F functions | N numbers | I interface | M memory map | T timing | O observation | S generation | P performance | G golden API | D decisions | Q open questions

- Service
- Loop
- Python
- Integer math
- **Spec**
- RTL
- Cells

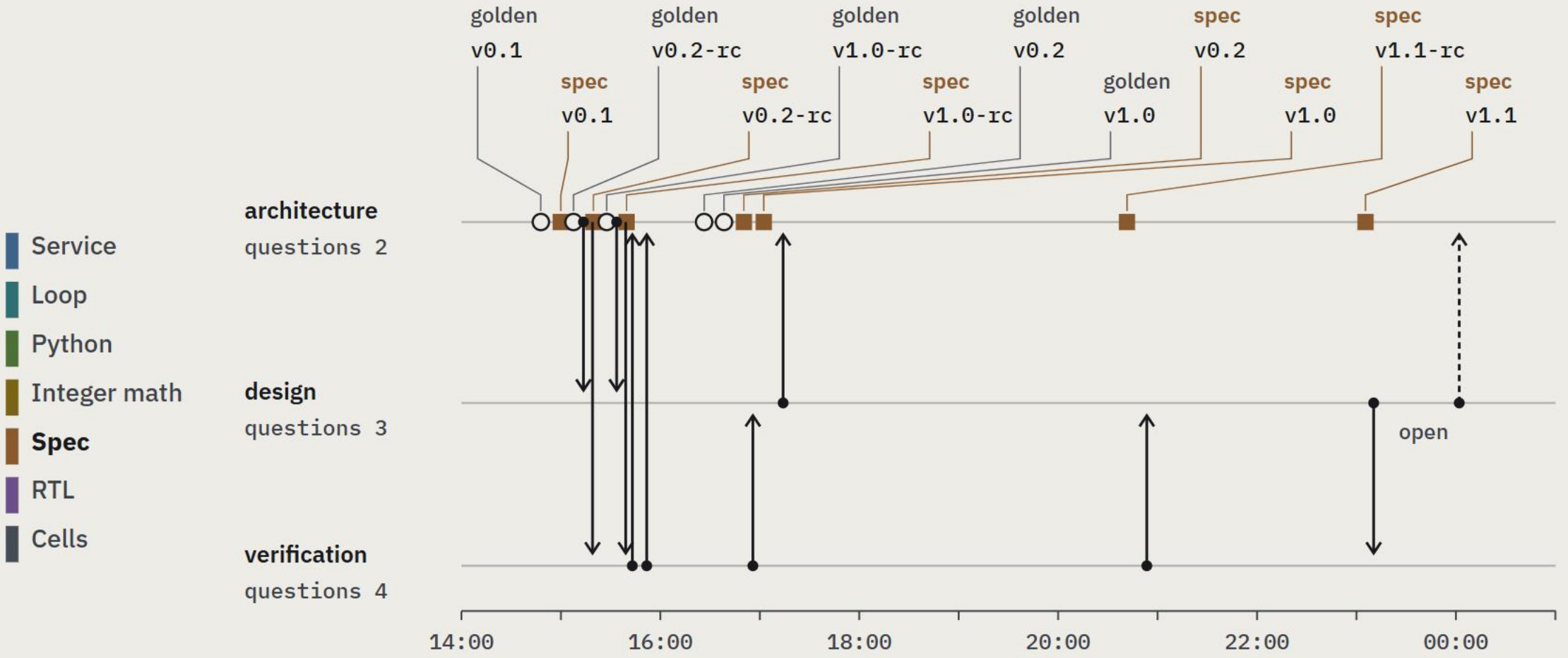
#	decision	rationale	rejected alternative
D9	**weights get one scale per row (per-row); requantize m · s per row too**	per-tensor gives the same NLL (−0.02%) but 2.6× the KL to the float distribution (5.4e−5 → 1.4e−4); argmax agreement 99.45% → 99.09%	per-tensor
D10	**residual is INT16, scale 2^−8**	INT8 (2^−5) has 1.5× the KL (8.3e−5), and the lm_head input absmax 4.19 exceeds the INT8 2^−5 range (±3.97)	INT8 residual
D12	**RMSNorm drops eps: even-shift normalisation + a 192-entry inverse-square-root table**	same NLL as a 768-entry table (10-bit mantissa) (+0.00%), KL 6.1e−5 vs 5.4e−5. The table is 1/4 the size	768-entry table; exact integer square root + division (a large circuit)
D13	**softmax: base-2 exponent (4 fraction bits, 16-entry table) + one reciprocal (an exact division) + UINT8 probabilities**	same NLL · KL as 8 fraction bits (256 entries) · 6 bits (64 entries) (5.35e−5). Scores are small (max |c·z| is small), so a coarse table is enough	256- · 64-entry tables; a division per element (T per head)
D18	**random numbers: one HW xorshift32, a SEED register, once per token**	golden and testbench must consume the same integer random sequence in the same order (M3 bit-exact rule). With the generator in HW, the testbench needs no handshake to supply random numbers. xorshift32 is only three XORs and three shifts	testbench supplies random numbers through a port (a handshake per position); LFSR one bit at a time (32 cycles)
D20	**integer CDF: r = (x · E) >> 32, the first token with cum > r**	makes an r close to uniform on [0, E) with no division or remainder (bias ≤ E/2^32 < 2^−12)	r = x mod E (a division); normalising the probabilities, then the CDF (27 divisions)

Functions, number formats, ports, memory map, timing and the golden-model interface. Design and verification can start together.

source spec-v1.1 decision table (SPEC.md, English rows translated)

Twelve spec and golden releases; answered spec questions became new versions

`00_shared/handoff/arch · architecture releases and questions`



`tool release.py · questions/`

When something was unclear, a team filed a question, and the answer became a new spec release.

`source facts.json spec, process.questions (release.py, questions/)`

not taped out · RTL simulation + sky130 synthesis

The spec became SystemVerilog modules

- Service
- Loop
- Python
- Integer math
- Spec
- **RTL**
  SystemVerilog modules
- Cells



built by **3 agent teams + 1 human** checks 2 simulators · Yosys checked by **verif (independent)** human design decisions ☐ **0**

The design team wrote 2,395 lines in 11 files

rtl-v1.1.1 · module hierarchy

- Service
- Loop
- Python
- Integer math
- Spec
- **RTL**
- Cells

2,395 lines of SystemVerilog · 11 files — rtl/*.sv

mg_soc_top	bus wrapper	181
└ **mg_top**	top	242
└ **mg_seq**	sequencer	726
└ **mg_mac16**	multiply lanes	48
└ **mg_rmsnorm**	RMS norm	297
└ **mg_rsqrt_rom**	lookup table	214
└ **mg_softmax**	softmax	290
└ **mg_exp2_rom**	lookup table	36
└ **mg_sample**	sampler	248
└ **mg_exp2_rom**	lookup table	36
└ **mg_rq**	requantize	74
└ **mg_sram_1rw ×4**	memory	39

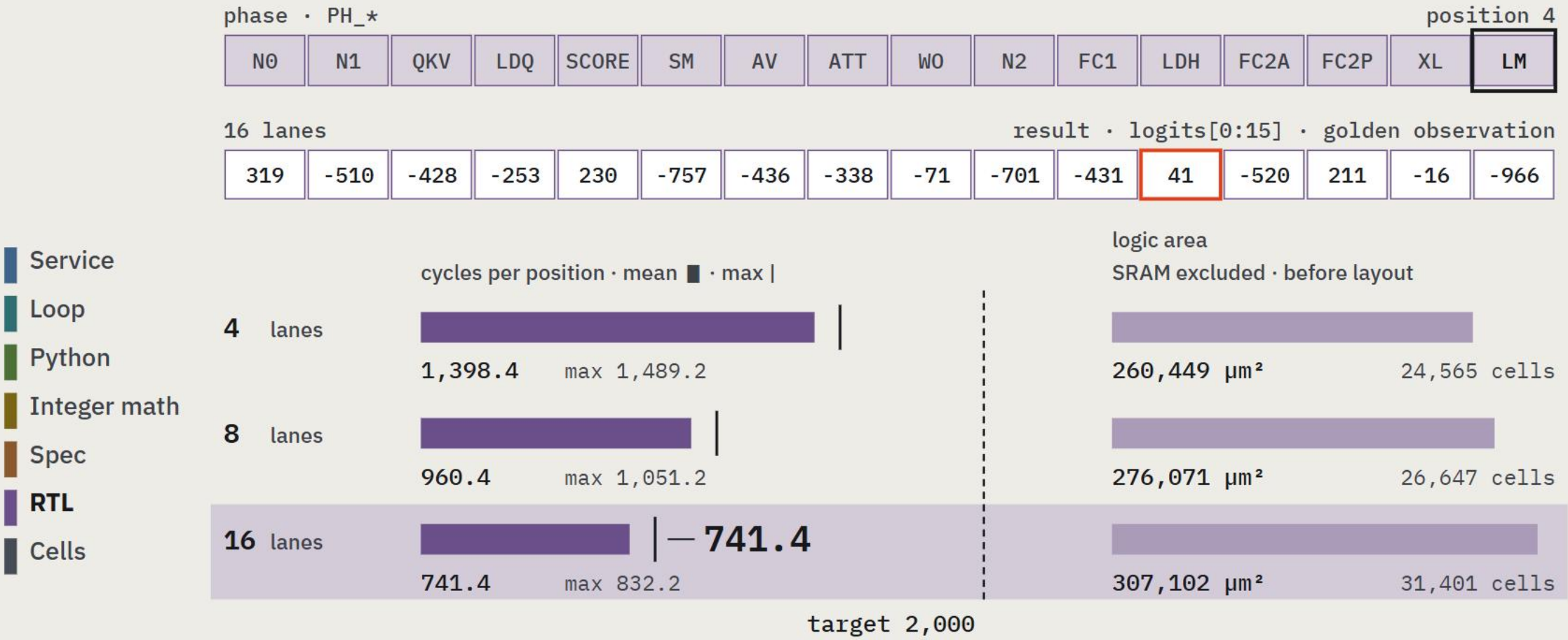
built by **design agent** tool gate.py · port_check.py

A bus wrapper, the top, a sequencer, and the blocks it drives: multiply lanes, RMS norm, softmax, sampler, requantizer, two tables, memory.

source facts.json rtl (gate.py, port_check.py)

Sixteen lanes average 741 cycles per position

mg_seq.sv · phase_e · the sequencer walks one position



tool dse.py

The sequencer walks one position step by step. Four, eight or sixteen lanes trade cycles for area; the target was 2,000 cycles.

source facts.json rtl DSE (dse.py)

Our letter’s score lands in memory as 41

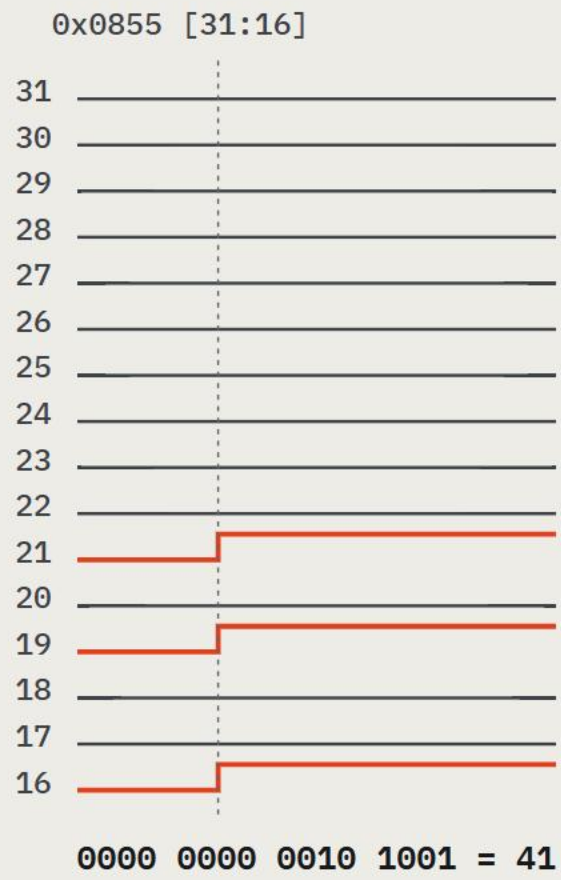
observation memory · LOGITS · 0x0850 ~ 0x085D · INT16 × 27

- Service
- Loop
- Python
- Integer math
- Spec
- **RTL**
- Cells

word		[31:16]	[15:0]
0x0850	0xFE02013F	-510	319
0x0851	0xFF03FE54	-253	-428
0x0852	0xFD0B00E6	-757	230
0x0853	0xFEAEFE4C	-338	-436
0x0854	0xFD43FFB9	-701	-71
0x0855	0x0029FE51	41	-431
0x0856	0x00D3FDF8	211	-520
0x0857	0xFC3AFFF0	-966	-16
0x0858	0xFFE4F8F9	-28	-1799
0x0859	0xFFA60021	-90	33
0x085A	0xFDAEFE29	-594	-471
0x085B	0xFD6CFCDC	-660	-804
0x085C	0xFE5AFF9E	-422	-98
0x085D	0x000000B0	0	176

our letter "l" · [11]

golden packing · not a waveform capture



Sixteen bits in an observation memory word. Drawn from the golden packing, not a waveform capture.

source facts.json thread hex, spec section 0 addresses

not taped out · RTL simulation + sky130 synthesis

Synthesis turns the RTL into standard cells

- Service
- Loop
- Python
- Integer math
- Spec
- RTL
- **Cells**
  sky130 standard cells



built by **3 agent teams + 1 human** checks 2 simulators · Yosys checked by **verif (independent)** human design decisions □ **0**

Only 16% of the logic is multiply lanes

SRAM not counted: 4 × mg_sram_1rw left as blackbox
logic only · synthesis, before place & route

30,150 standard cells

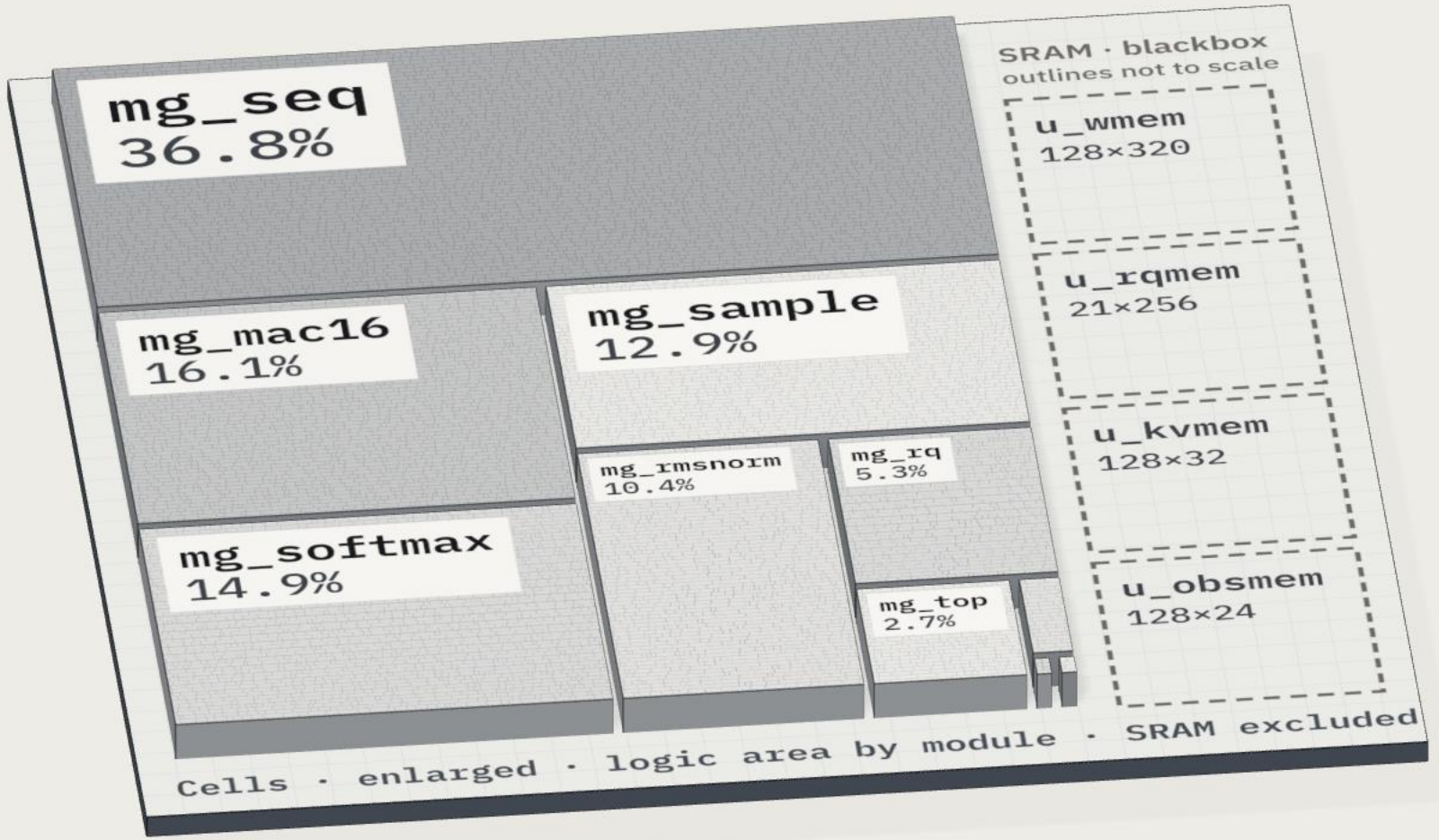
0.313 mm² logic area

4,943 flip-flops

0 latches

share of logic area (own area per module)

mg_seq	36.8%
mg_mac16	16.1%
mg_softmax	14.9%
mg_sample	12.9%
mg_rmsnorm	10.4%
mg_rq	5.3%
top + ROMs	3.5%



not taped out · RTL simulation + sky130 synthesis

Verification checks the design against the spec

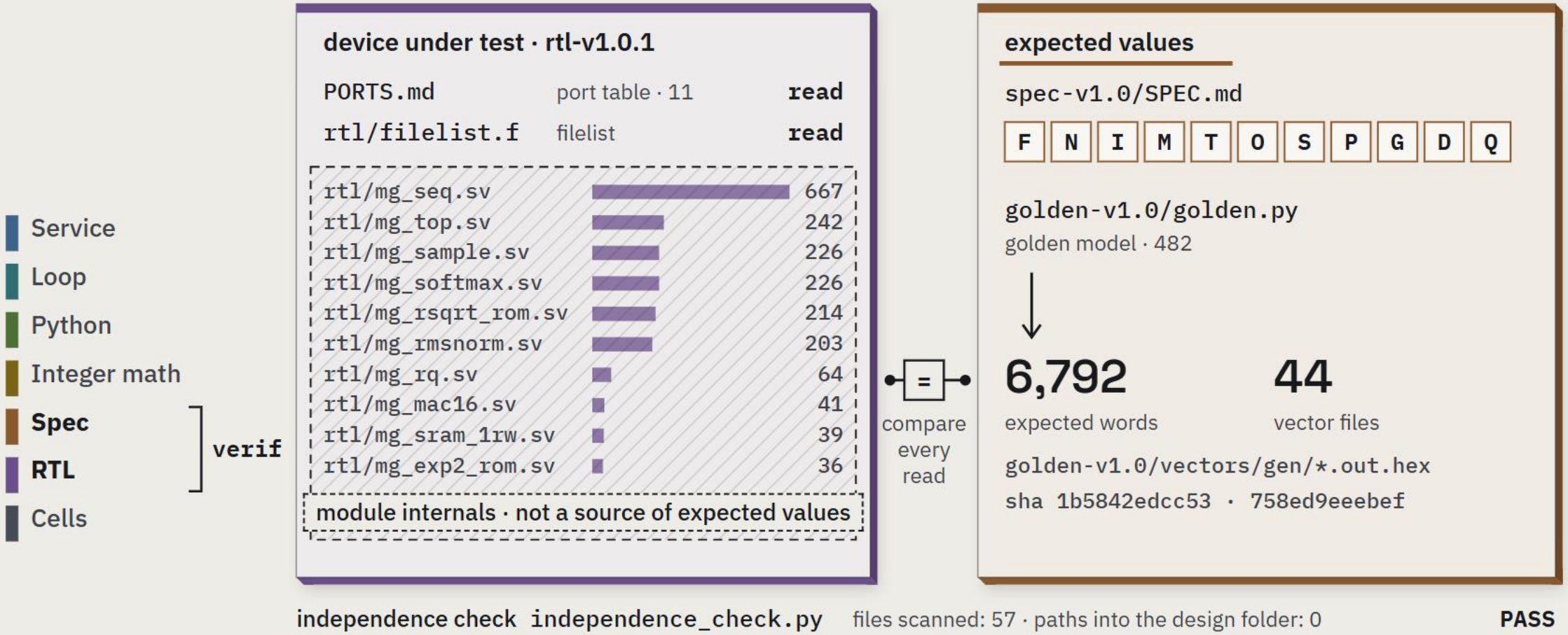
Service
Loop
Python
Integer math
Verification
Spec
RTL
SystemVerilog modules
Cells



built by **3 agent teams + 1 human** checks 2 simulators · Yosys checked by **verif (independent)** human design decisions □ **0**

Every expected value came from the spec and golden model

Verification · independent



independence check `independence_check.py` files scanned: 57 · paths into the design folder: 0 **PASS**

built by **verif agent (independent)** tool `independence_check.py`

Verification never looked inside the RTL to decide the right answer. It did read the port table and file list.

source `independence_check.py`

Two simulators, over 50,000 reads each, zero mismatches

M3 regression · two simulators · rtl-v1.0.1 · golden-v1.0



tool regress.py · cov_merge.py

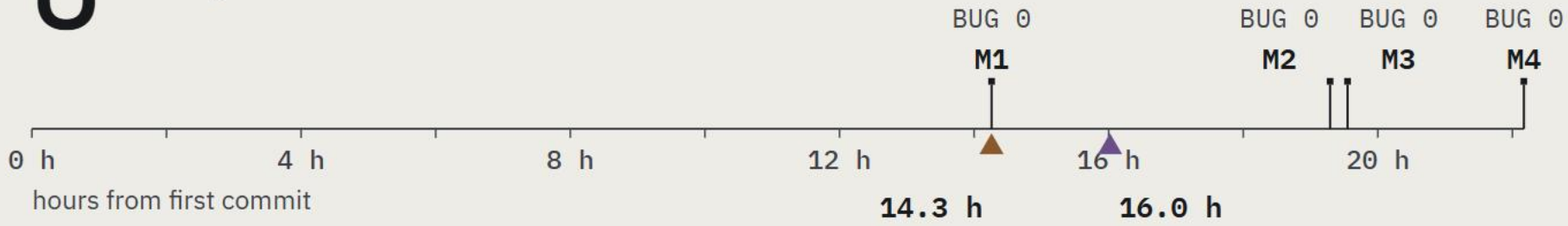
The same tests on two different simulators. All 65 coverage bins were hit.

source facts.json verif M3 (regress.py, cov_merge.py)

Milestone checks found no RTL bugs; catches came earlier

What verification caught

0 RTL bugs found at milestone checks



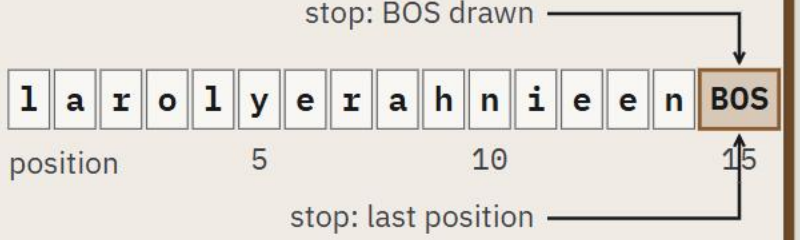
- Service
- Loop
- Python
- Integer math
- Spec
- **RTL**
- Cells

verif

spec rc review · verif Spec · 14.3 h

Q-verif-002 · spec-v1.0-rc

end-of-name corner case



vector lengths before: 0–8, 16

added to golden: stress15 · 15

design gate · rq smoke RTL · 16.0 h

20_design/00_META/ledger.jsonl:18

a 1-bit signed constant



meant: +1

fixed line · rtl-v1.1.1 · mg_rq.sv:54

```
qi = (T_W + 1)'(q_q) + (T_W + 1)'(1);
```

Spec review found an untested corner case where two stop rules meet; the design gate caught a constant that meant minus one.

source facts.json verif catches

Tests caught 49 of 76 deliberately broken designs

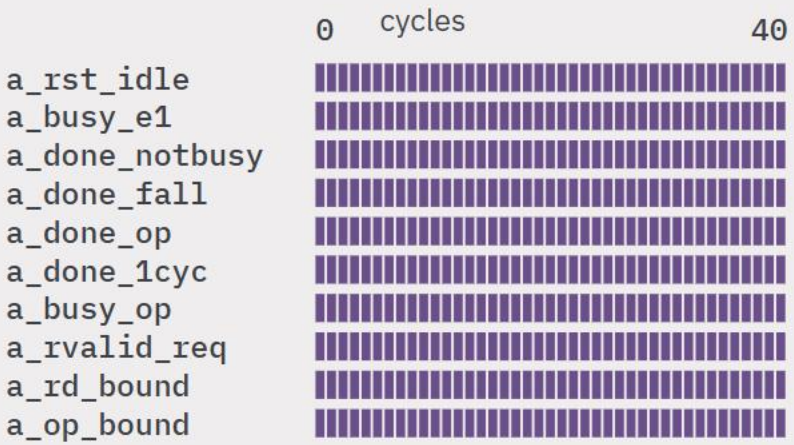
Formal · mutation

- Service
- Loop
- Python
- Integer math
- **Spec**
- **RTL**
- Cells

verif

bounded formal · port SVA

every input (solver)



M1 · rtl-v0.1.0	PASS · 40 · 13.1 s
M2 · rtl-v0.2.1	PASS · 40 · 643.0 s
M3 · rtl-v1.0.1	not judged · out of memory

mutation · first RTL · rtl-v0.1.0

49 / 76 mutants · icarus · 3.9 min



operator

- 49 caught by tests
- 17 formal-only code · simulation never runs it
- 10 likely changes nothing · not proven

+2 did not compile

tool **formal.py · mutate.py**

Mutation on the first RTL only; most survivors sit in formal-only lines. Bounded formal ran on milestones 1 and 2, 40 cycles deep.

source facts.json verif formal, mutation (formal.py, mutate.py)

Back at the top, RTL and golden names match bit for bit

Back to the top · Service

- **Service**
- Loop
- Python
- Integer math
- Spec
- RTL
- Cells

RTL simulation NAME · hex

seed 42 · 20 / 44

anas	00 0d 00 12
chena	02 07 04 0d 00
melelan	0c 04 0b 04 0b 00 0d
karilen	0a 00 11 08 0b 04 0d
janan	09 00 0d 00 0d
caria	02 00 11 08 00
carila	02 00 11 08 0b 00
arana	00 11 00 0d 00
carinn	02 00 11 08 0d 0d
anna	00 0d 0d 00
arady	00 11 00 03 18
jaran	09 00 11 00 0d
alaran	00 0b 00 11 00 0d
jarie	09 00 11 08 04
ahrii	00 07 11 08 08
alari	00 0b 00 11 08
ratana	11 00 13 00 0d 00
lana	0b 00 0d 00
dian	03 08 00 0d
dosken	03 0e 12 0a 04 0d

bit-exact

golden model NAME · hex

seed 42 · 20 / 44

anas	00 0d 00 12
chena	02 07 04 0d 00
melelan	0c 04 0b 04 0b 00 0d
karilen	0a 00 11 08 0b 04 0d
janan	09 00 0d 00 0d
caria	02 00 11 08 00
carila	02 00 11 08 0b 00
arana	00 11 00 0d 00
carinn	02 00 11 08 0d 0d
anna	00 0d 0d 00
arady	00 11 00 03 18
jaran	09 00 11 00 0d
alaran	00 0b 00 11 00 0d
jarie	09 00 11 08 04
ahrii	00 07 11 08 08
alari	00 0b 00 11 08
ratana	11 00 13 00 0d 00
lana	0b 00 0d 00
dian	03 08 00 0d
dosken	03 0e 12 0a 04 0d

20 / 20 names identical

44 names · icarus · verilator · mismatch 0

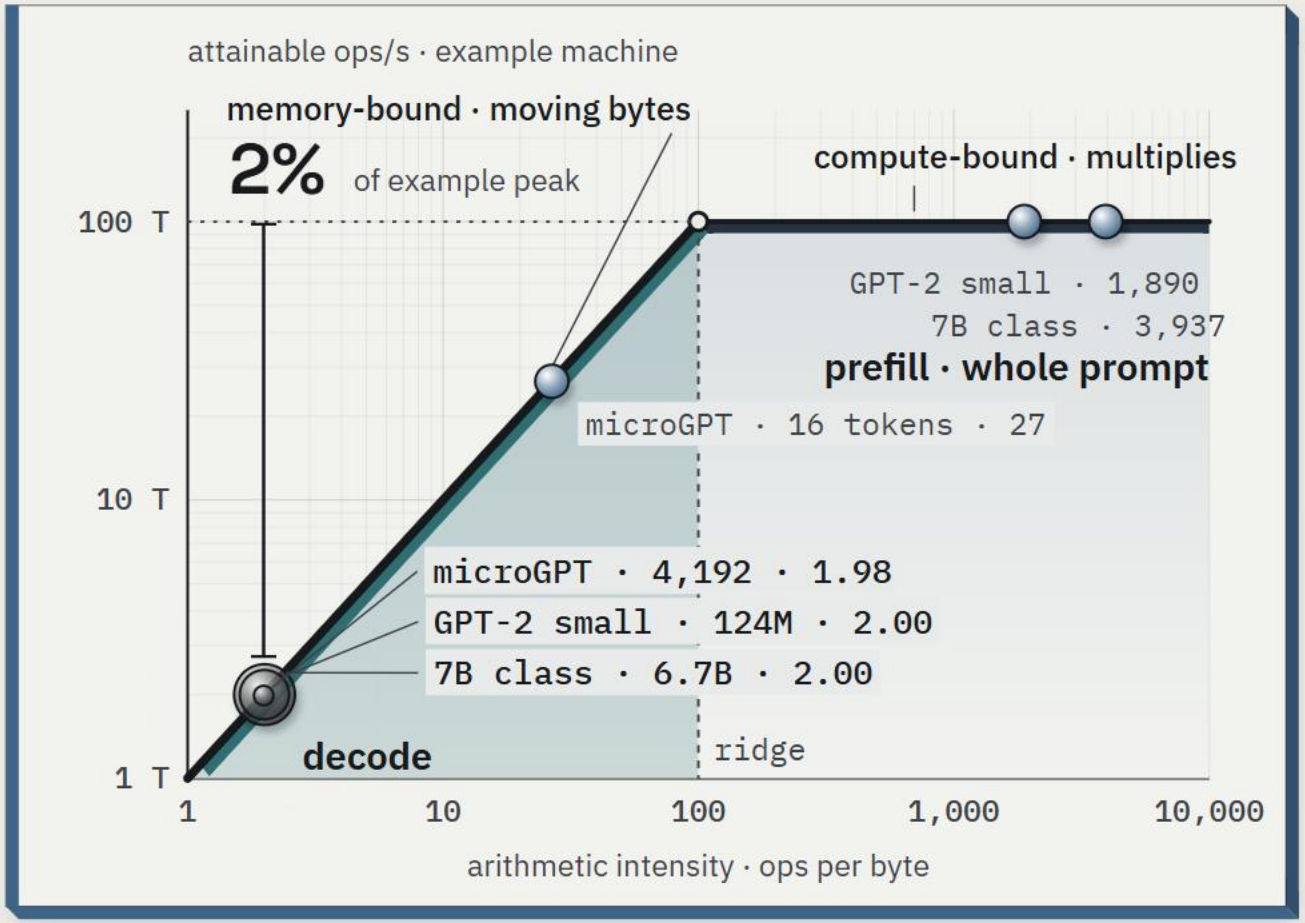
The RTL in simulation and the golden model each generated these names from the same seed.

source facts.json names

At real scale, moving data limits generation, not multiplying

Real scale · roofline

computed from public definitions, not measured



decode · one token · batch 1

one multiply-add per weight byte → 2 ops / byte

INT8 weights and KV · full context

model	ops / token	bytes / token	ops / byte
microGPT 4,192	8.0 K	4.0 KB	1.98
GPT-2 small 124M	285 M	143 MB	2.00
7B class 6.7B	14.3 G	7.1 GB	2.00

weights · KV cache

ridge · example machine

100 TOPS ÷ 1 TB/s = 100 ops / byte

computed · not measured build_backup.py

When a model writes one token at a time, weights and the key-value cache must be moved for every token.

source build_backup.py: computed from public definitions, not measured

not taped out · RTL simulation + sky130 synthesis

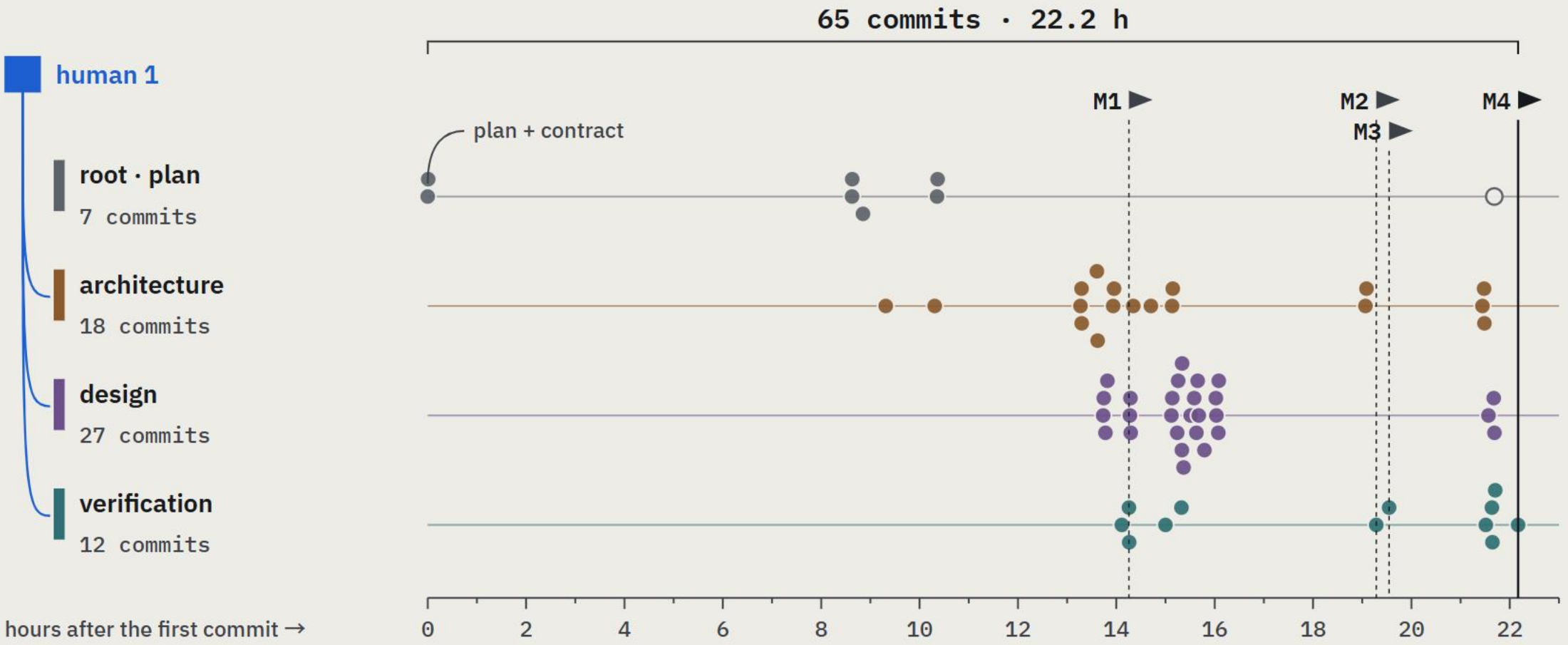
Three agent teams and one human built it



built by **3 agent teams + 1 human** checks 2 simulators · Yosys checked by **verif (independent)** human design decisions ☐ **0**

65 commits in 22.2 hours, from plan to milestone four

git log · how it was built



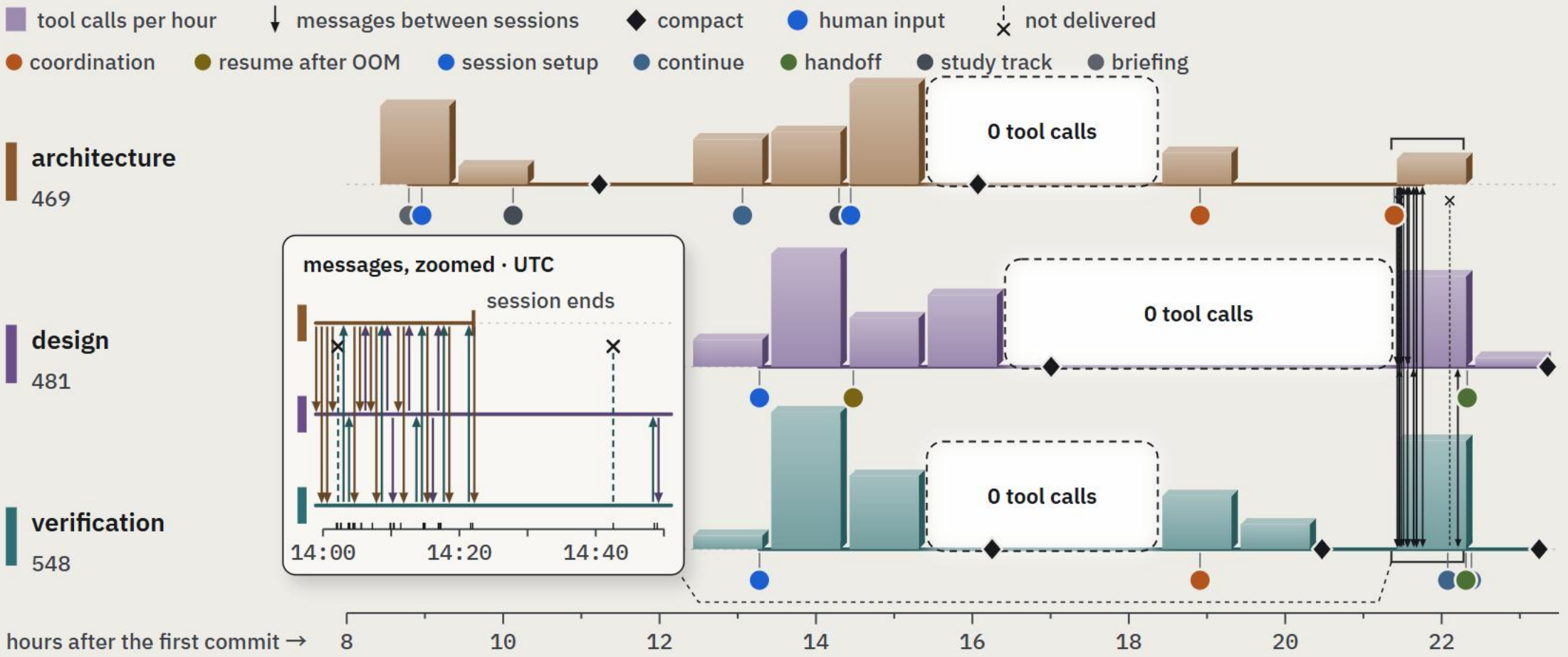
agent teams **3** · human **1** · commits **65** · **22.2 h**

From a plan and a contract to the fourth milestone's top-level tests passing.

source facts.json
process.commits_by_hour,
milestone_commits

Every team's day had hours with zero tool calls

wf_usage.json · session records · one lane per team



tool calls **1,498** · messages between sessions **30 (28 delivered, 2 failed)** **+4** to own sub-agents · compact **7** · human input **16**

Bars are tool calls per hour, diamonds compacts, dots human inputs. All 30 messages between sessions fall after 14:00 UTC (zoomed inset); 2 were not delivered.

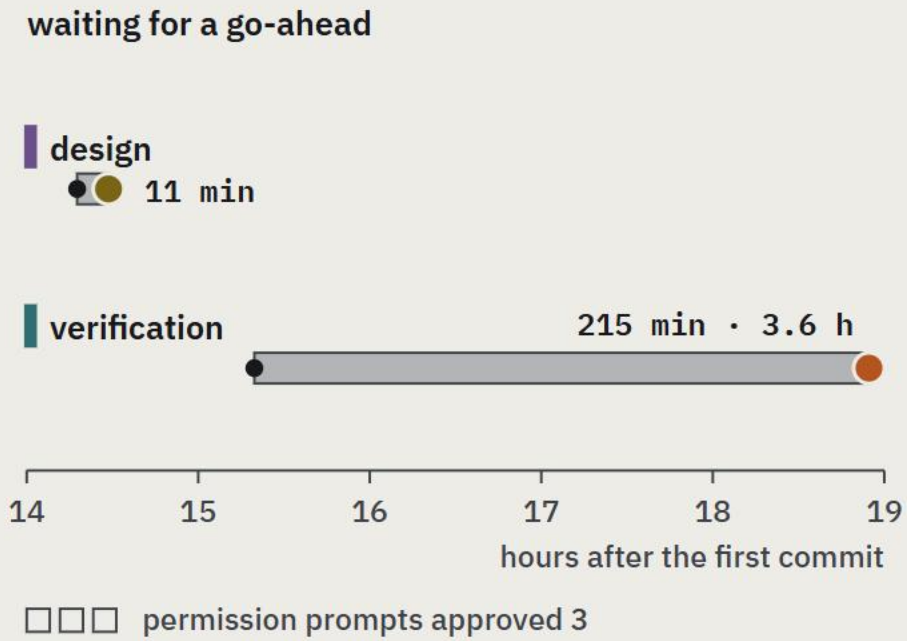
source wf_usage.json autonomous_branches, human_coding rows

None of the human's 16 inputs was a design decision

wf_usage_human_coding.json · the human's 16 inputs, by kind

design decision	0	none
coordination	3	● ● ●
resume after OOM	1	●
session setup	4	● ● ● ●
continue	3	● ● ●
handoff	2	● ●
study track	2	● ●
briefing	1	●

operating interventions 4



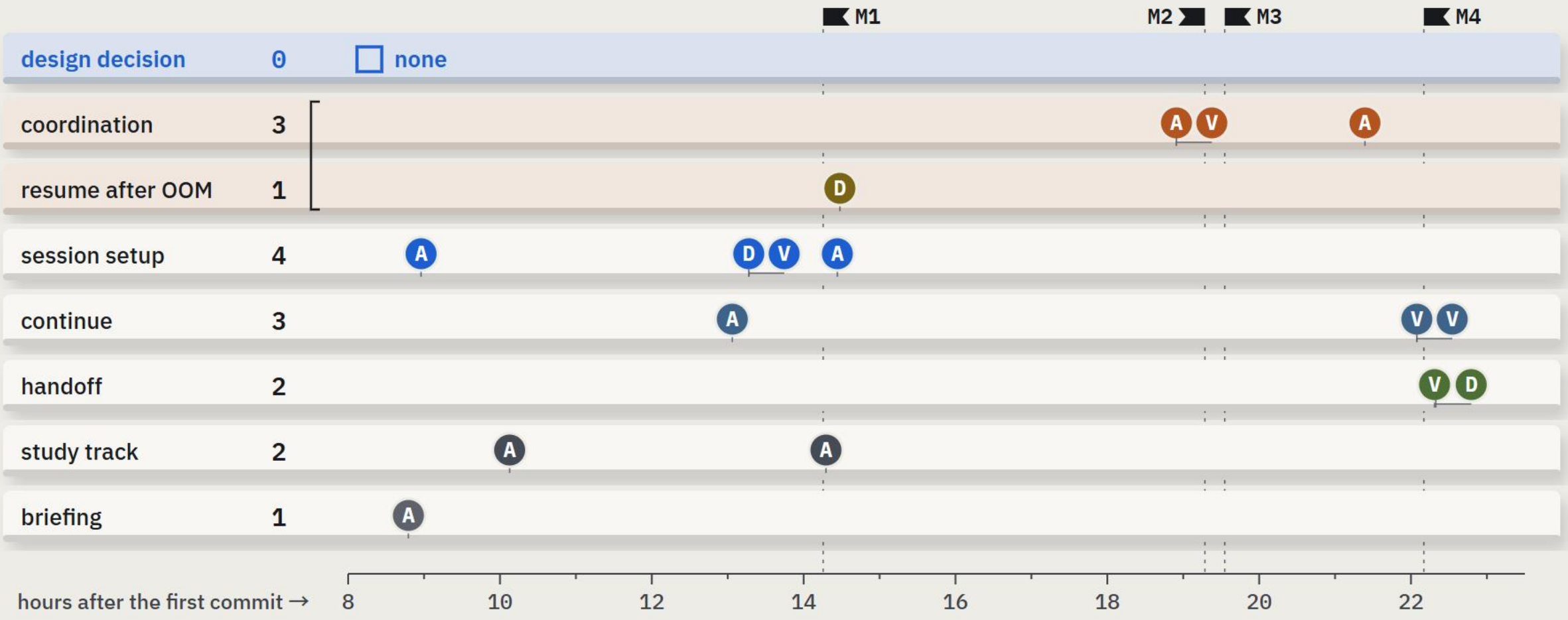
design decision **0** · operating interventions **4 (coordination 3, resume after OOM 1)** · waiting for a go-ahead **3.6 h**

Four were operating interventions: three relayed messages or broke a wait, one allowed a rerun after memory ran out.

source wf_usage_human_coding.json summary

wf_usage_human_coding.json · 16 human inputs by kind and time

Setups came early, coordination later, handoffs at the very end



A architecture D design V verification ☐ ☐ ☐ permission prompts approved, not placed in time: 3

kinds coded by Claude from the records; not yet reviewed by a human; input text not shown

Ledgers noted one human intervention; session records show four

self-report vs record

ledger · self-report

*/00_META/ledger.jsonl

human interventions

1



sub-agent type

mg-rtl-writer



×6

≠

≠

session records

sessions/*.jsonl · 5

human interventions

4



operating: coordination 3 · resume after OOM 1

sub-agent type

general-purpose



×6

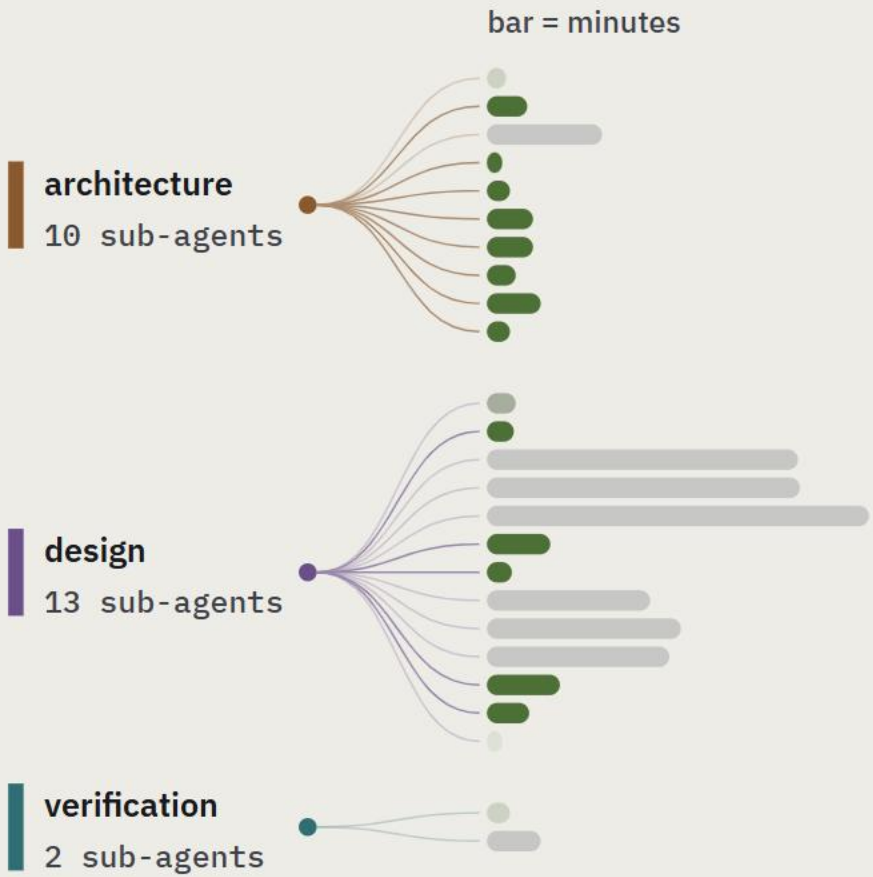
count from the records **1 vs 4 · 6 mg-rtl-writer vs 6 general-purpose**

Six jobs credited to a custom agent all ran as general-purpose. Self-reports alone would hide both, so we count from the records.

source facts.json process.human_interventions_total, human_coding summary

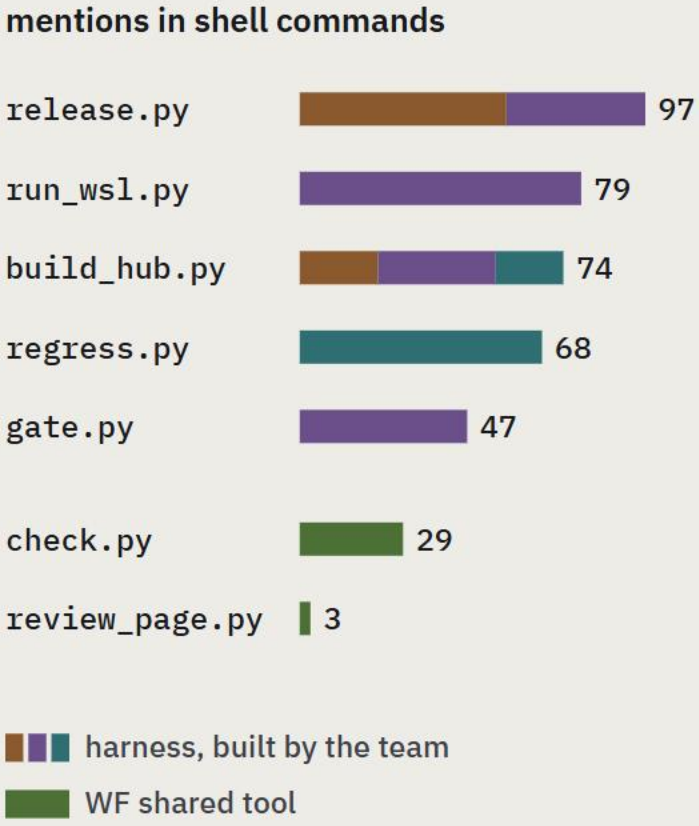
25 sub-agents ran; 13 were independent reviews

wf_usage.json · who did the work



wf-second-look	13
general-purpose	8
wf-project-scout	2
wf-researcher	1
wf-wrap-refiner	1

13 independent reviews



sub-agents **25** · independent reviews **13** · mentions in shell commands **release · run_wsl · build_hub · regress · gate**

The scripts named most often in shell commands were harness scripts each team built for itself at the start.

source wf_usage.json subagents_by_type, scripts_top

Reviews found most in the spec; questions came from verification

WF_IN_PRACTICE_cases.md · three observations

n = 1 · one run · observations, not effects

1 reviews found the most in the spec

spec readiness reviews

M1	M2-rc	M3-rc	v1.1-rc
✓	✓	✓	✓

4/4 flagged

RTL release reviews

v0.1.0	v0.2.0	v1.0.0	v1.1.0
0	0	0	0

wrong 0/4

2 questions on spec drafts came from verification

spec-v0.2-rc · rc review

verification	design
1	0

spec-v1.0-rc · rc review

verification	design
1	0

questions

verification 2 · design 0

3 the human coordinated, did not design

human inputs 16



design decision 0

- coordination 3
- resume after OOM 1

Three observations from a single run. The human coordinated sessions and restarted work after memory ran out.

source WF_IN_PRACTICE.md observations, cases C02 C04 C24 C45

One run, one model family: observations, not effects

`n = 1`

One topic, one autonomous run.

A run with a human making the decisions would give a comparison.

`same model family`

Independent reviews share the model's biases.

A reviewer that did not make the work is independent in context, not in model.

`analyst = executor family`

Claude did this analysis and the input coding.

A human must check the coding against the input text before it is a claim.

`approximate classes`

Shell-command and failure classes are regular expressions.

Read those numbers as tendencies.

`snapshot`

Session records keep growing.

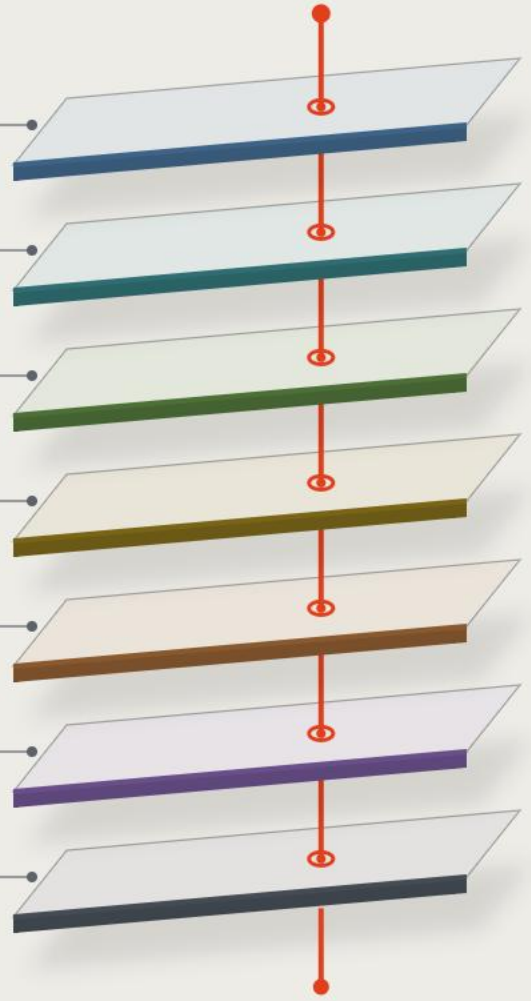
Claims use the three closed team sessions only.



observations, not effects

One token, seven layers, each in its own language

layer	what it is	the thread at this layer
Service	a chat API streams tokens	`token = "l"`
Loop	forward · sample · append	`pos 4 → sample`
Python	199 lines, float	`p(l) = 4.1% · rank 5`
Integer math	INT8 weights, fixed scales	`logit_q = 41 (INT16)`
Spec	the contract both teams build to	`41 · 2^-8 = 0.160`
RTL	SystemVerilog modules	`OBS 0x0855 = 0x0029FE51`
Cells	sky130 standard cells	`q[15:0] = 0000 0000 0010 1001`



the thread: one token, the “l” in karilen

source scenes/style-frame/data.js thread, the Hero plate tags

Where does a token come from?

video

WF, the workflow that ran these agent teams

youtu.be/e6XbvrdJfWc

next run

a human makes the decisions

